Software Re-Engineering

Lecture 16

Reverse engineering III – IDA Displays II

Instructor:

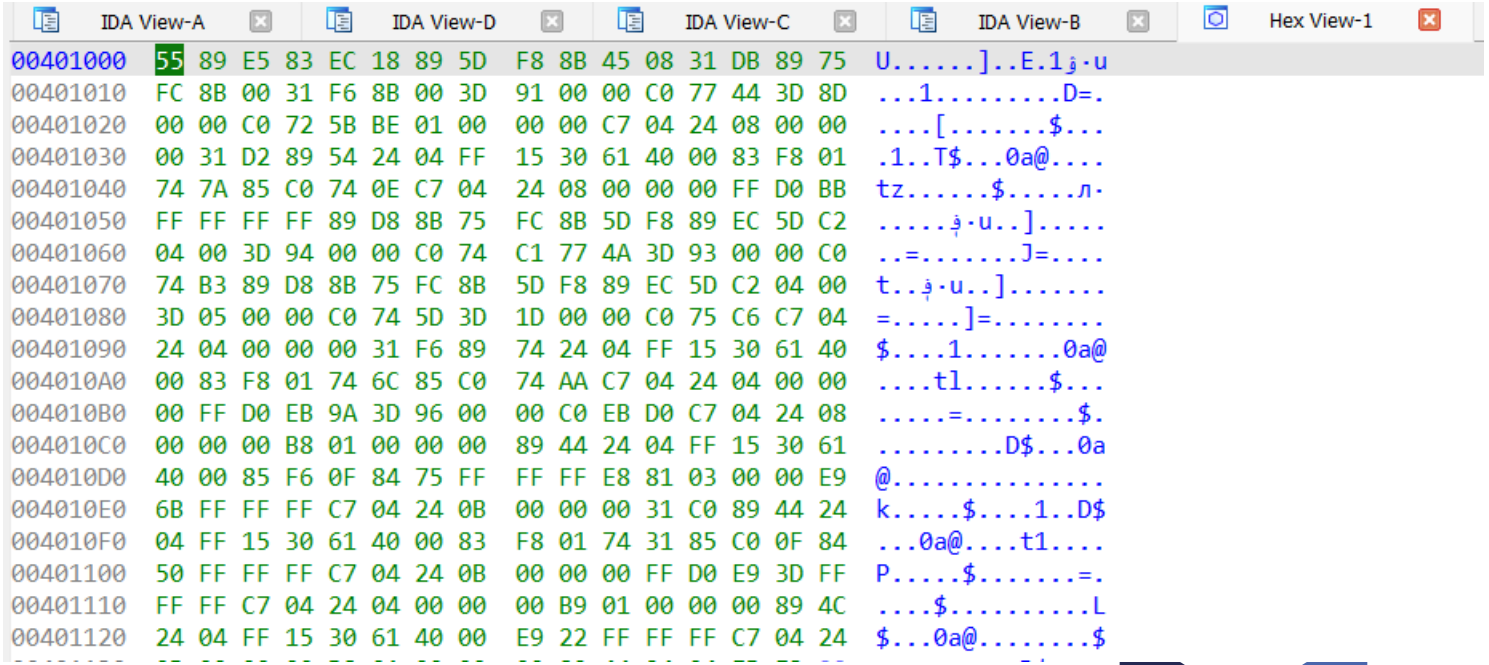
Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

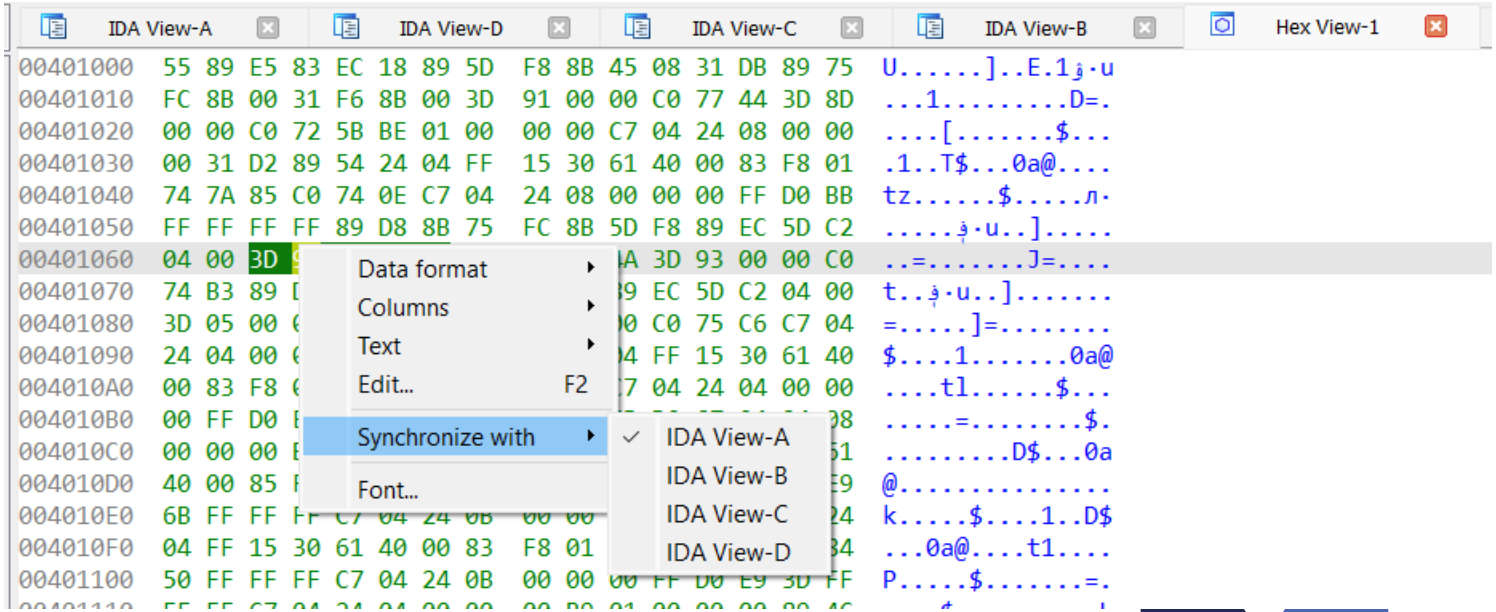
SE Fall 2020

The Secondary IDA Displays

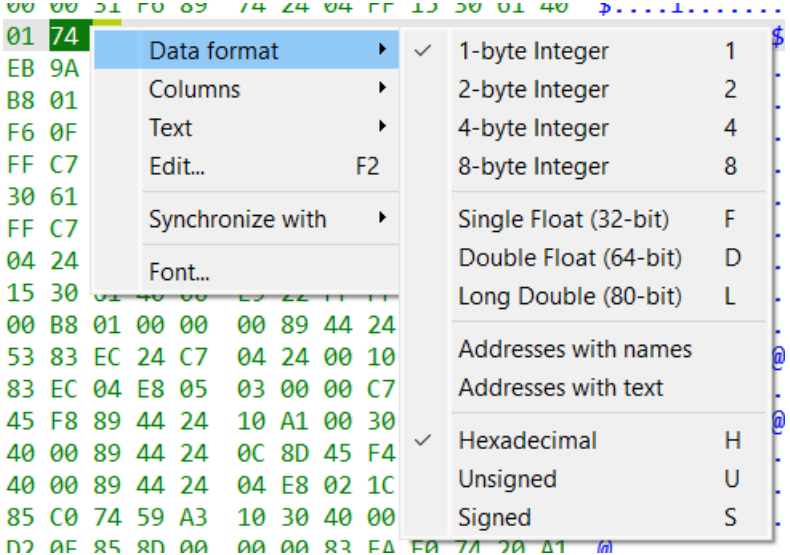
By default, the **Hex View window** provides a standard hex dump of the program content with 16 bytes per line and ASCII equivalents displayed alongside.



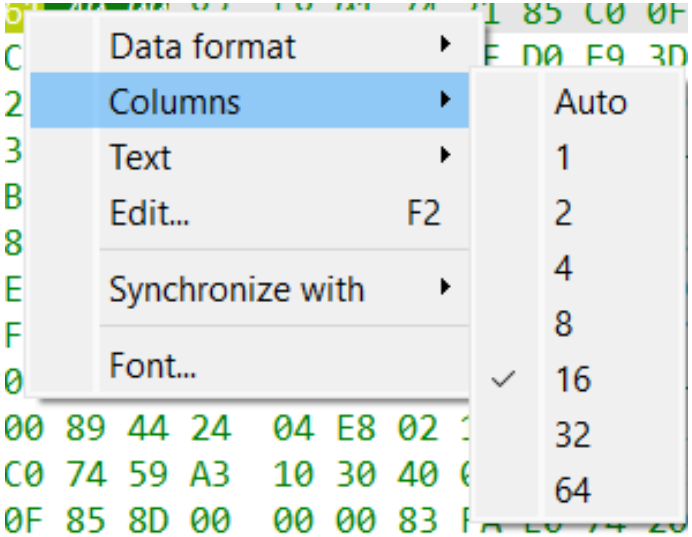
By default, the first Hex window is synchronized with the first disassembly window. When a disassembly view is synchronized with a hex view, scrolling in one window causes the other window to scroll to the same location (same virtual address).



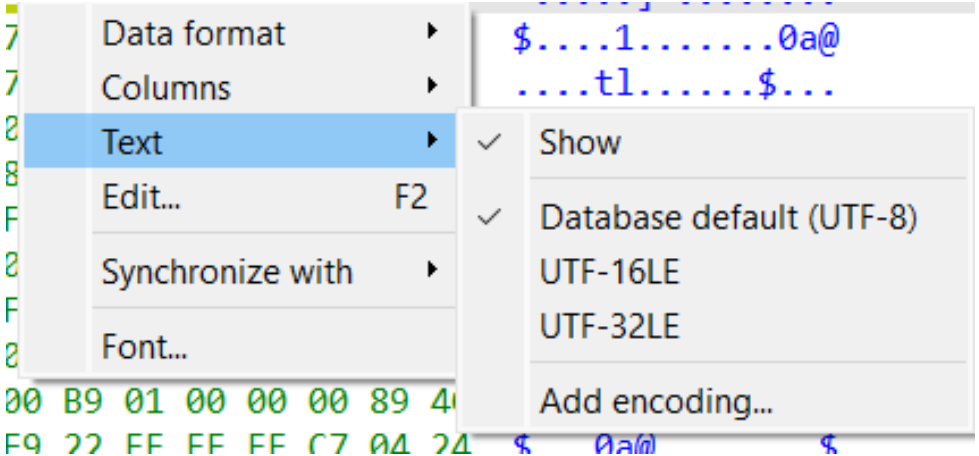
The Data Format menu item allows you to select from a variety of display formats such as 1-, 2-, 4-, or 8-byte hexadecimal; signed decimal; or unsigned decimal integers and various floating point formats.



The Columns menu option allows you to change the number of columns used in the display

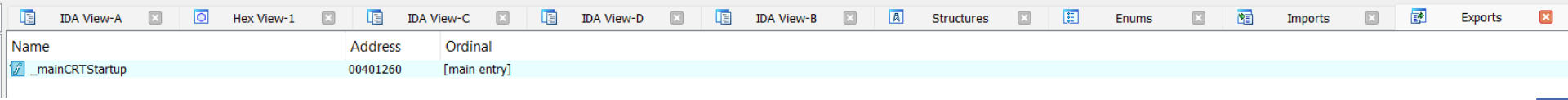


- Text option allows you to turn the text dump on and off.



Exports Window

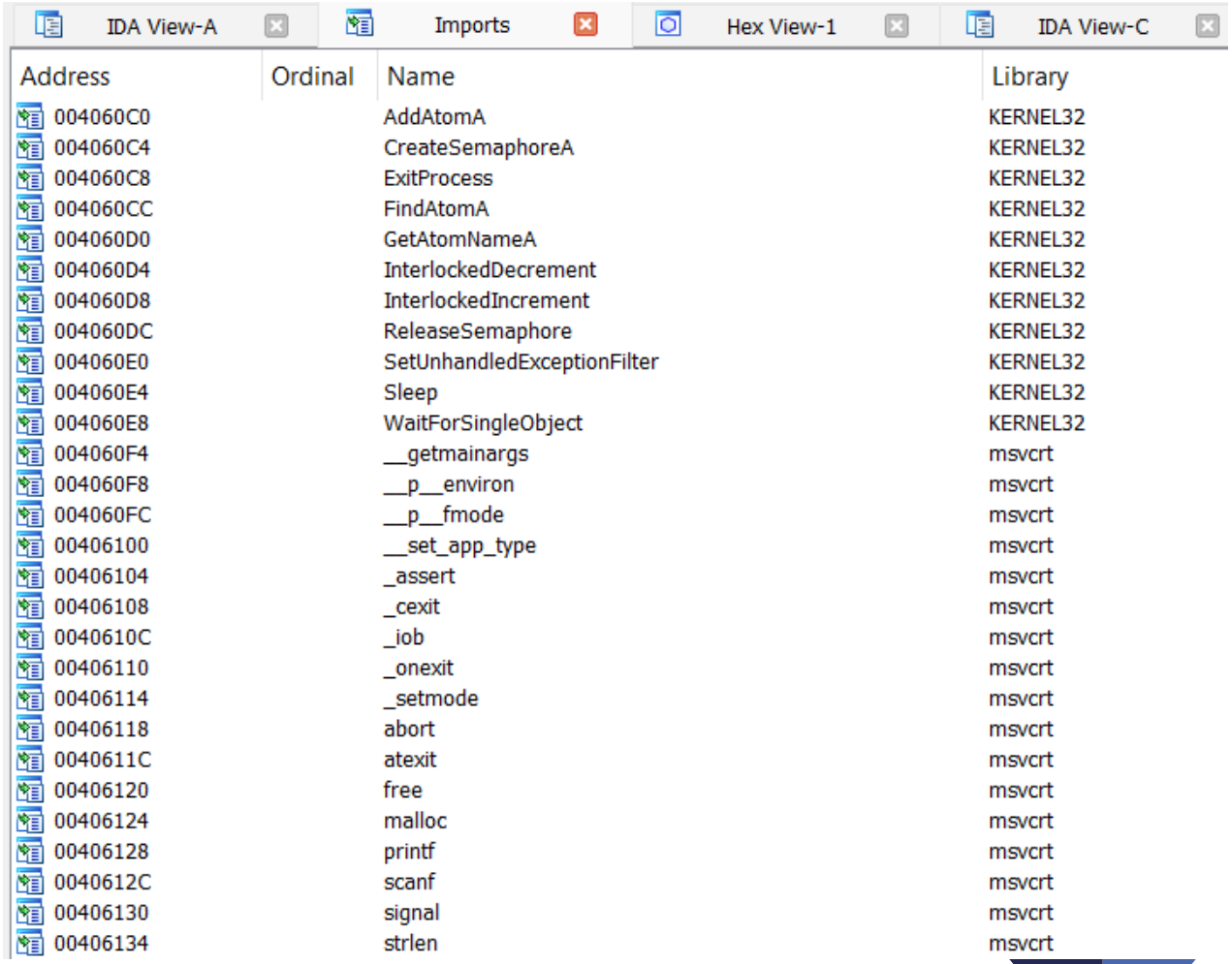
- The Exports window lists the entry points into a file.
- These include the program's execution entry point, as specified in its header section, along with any functions and variables that the file exports for use by other files.
- Exported functions are commonly found in shared libraries such as Windows DLL files.



- Exports window always contains at least one entry: the program's execution entry point. IDA names this entry point start

The Imports Window

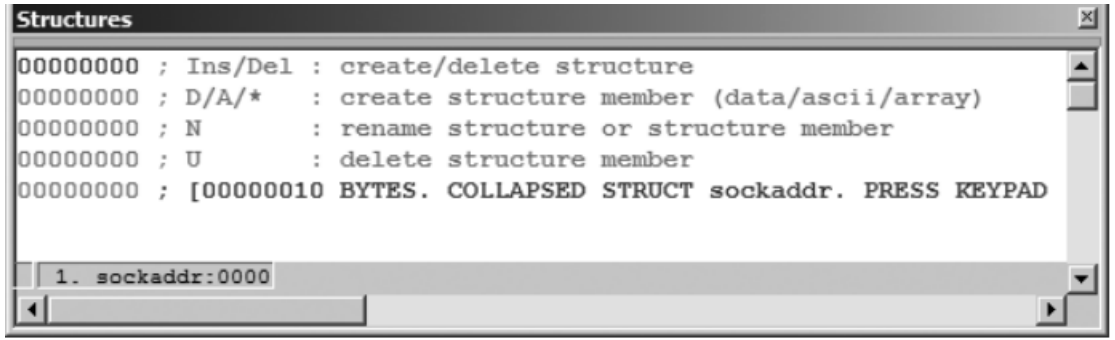
- The Imports window is a counterpart to the Exports window. It lists all functions that are imported by the binary being analyzed.
- The Imports window is relevant only when a binary makes use of shared libraries.
- Statically linked binaries have no external dependencies and therefore no imports.
- Each entry in the Imports window lists the name of an imported item (function or data) and the name of the library that contains that item.



Address	Ordinal	Name	Library
004060C0		AddAtomA	KERNEL32
004060C4		CreateSemaphoreA	KERNEL32
004060C8		ExitProcess	KERNEL32
004060CC		FindAtomA	KERNEL32
004060D0		GetAtomNameA	KERNEL32
004060D4		InterlockedDecrement	KERNEL32
004060D8		InterlockedIncrement	KERNEL32
004060DC		ReleaseSemaphore	KERNEL32
004060E0		SetUnhandledExceptionFilter	KERNEL32
004060E4		Sleep	KERNEL32
004060E8		WaitForSingleObject	KERNEL32
004060F4		__getmainargs	msvcrt
004060F8		__p__environ	msvcrt
004060FC		__p__fmode	msvcrt
00406100		__set_app_type	msvcrt
00406104		_assert	msvcrt
00406108		_cexit	msvcrt
0040610C		_iob	msvcrt
00406110		_onexit	msvcrt
00406114		_setmode	msvcrt
00406118		abort	msvcrt
0040611C		atexit	msvcrt
00406120		free	msvcrt
00406124		malloc	msvcrt
00406128		printf	msvcrt
0040612C		scanf	msvcrt
00406130		signal	msvcrt
00406134		strlen	msvcrt

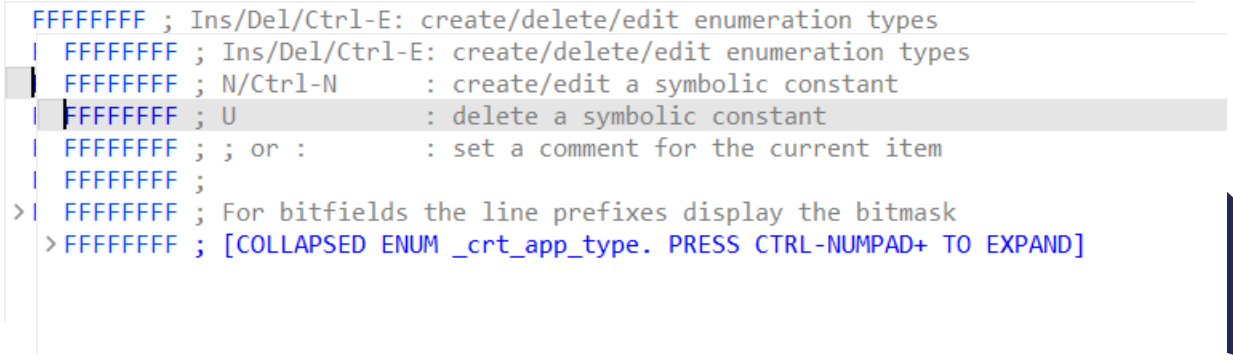
The Structures Window

- The Structures window is used to display the layout of any complex data structures, such as C structs or unions, that IDA determines are in use within a binary.
- During the analysis phase, IDA consults its extensive library of function type signatures in an attempt to match function parameter types to memory used within the program.



The Enums Window

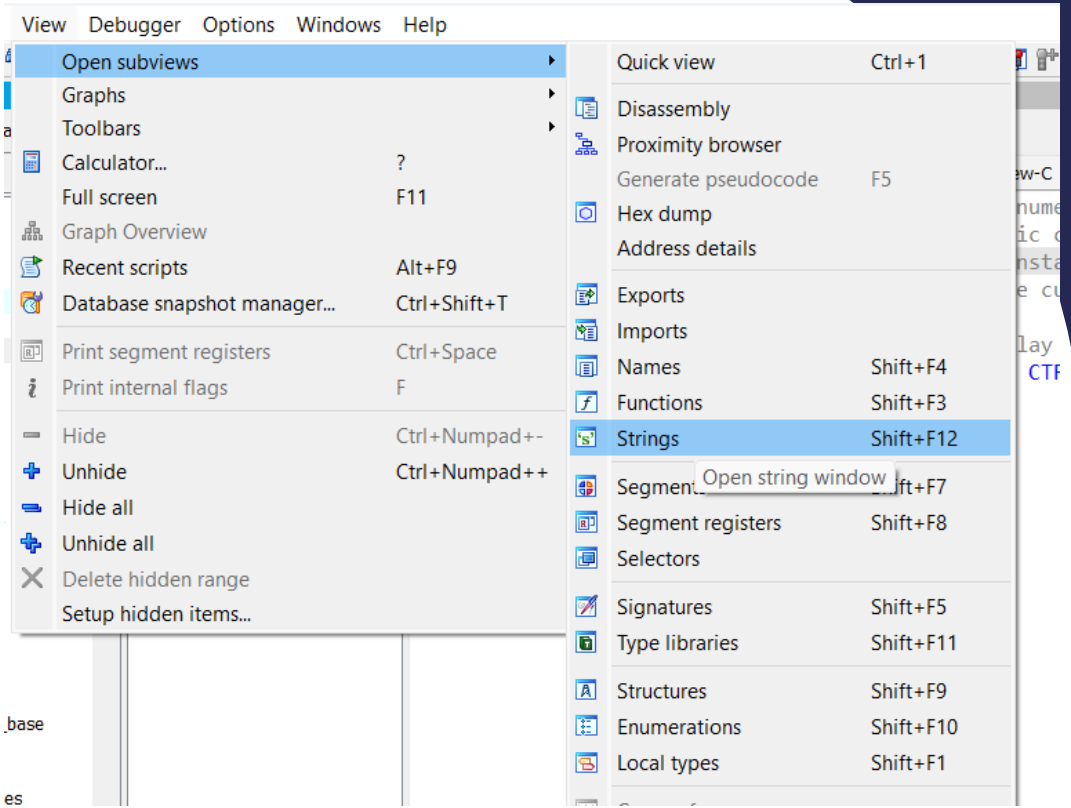
- The Enums window is somewhat similar to the Structures window. When IDA detects the use of a standard enumerated datatype (C enum), that datatype will be listed in the Enums window.
- You can make your disassemblies far more readable by using enums in place of integer constants.
- Like the Structures window, the Enums window offers facilities for defining your own enumerated types that you can use with your disassembled binaries



The Tertiary IDA Displays

- The last windows that we will discuss are those that IDA does not open by default.
- Each of these windows is available via View->Open Subviews, but they tend to provide information to which you may not require immediate access and are thus initially kept out of the way

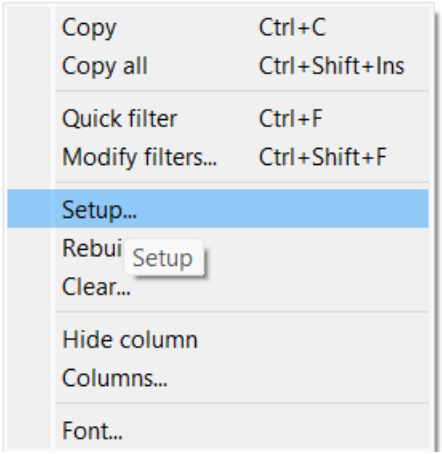
- The purpose of the Strings window is to display a list of strings extracted from a binary along with the address at which each string resides.
- double-clicking any string listed in the Strings window causes the disassembly window to jump to the address of the selected string.

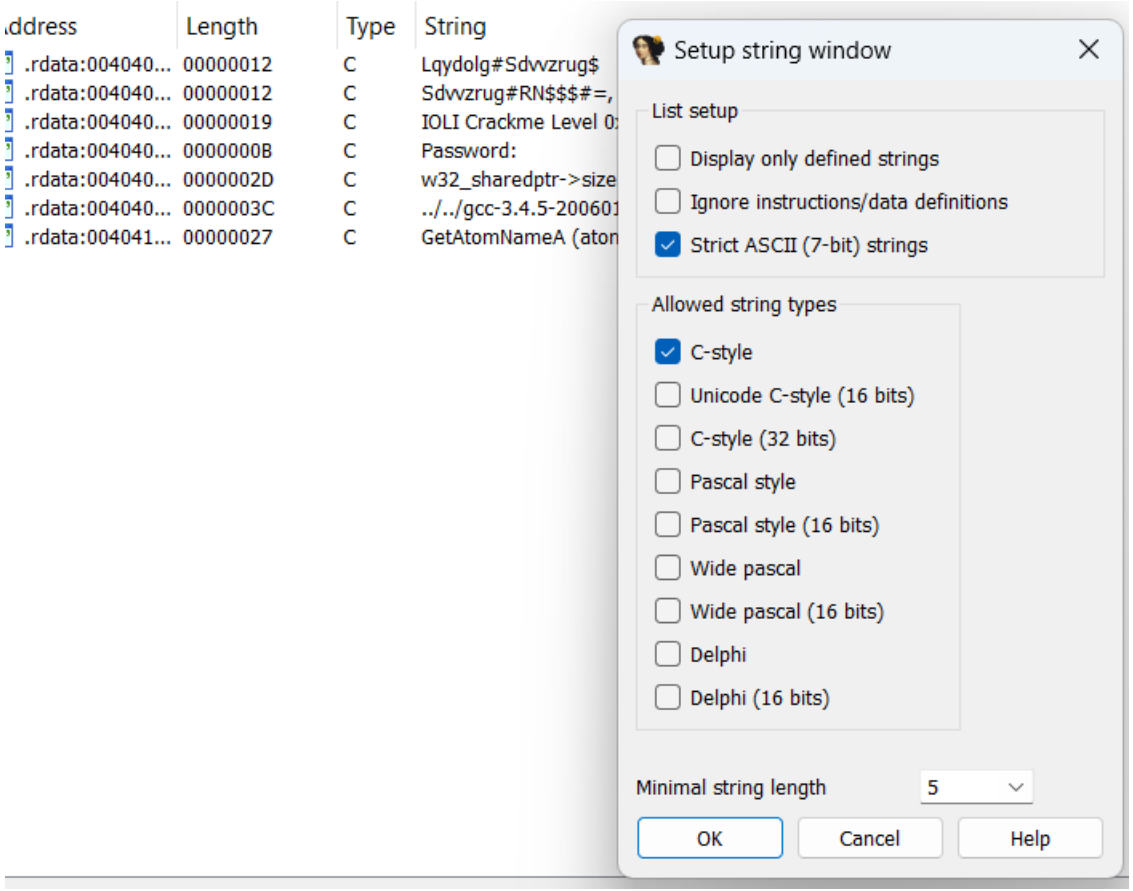


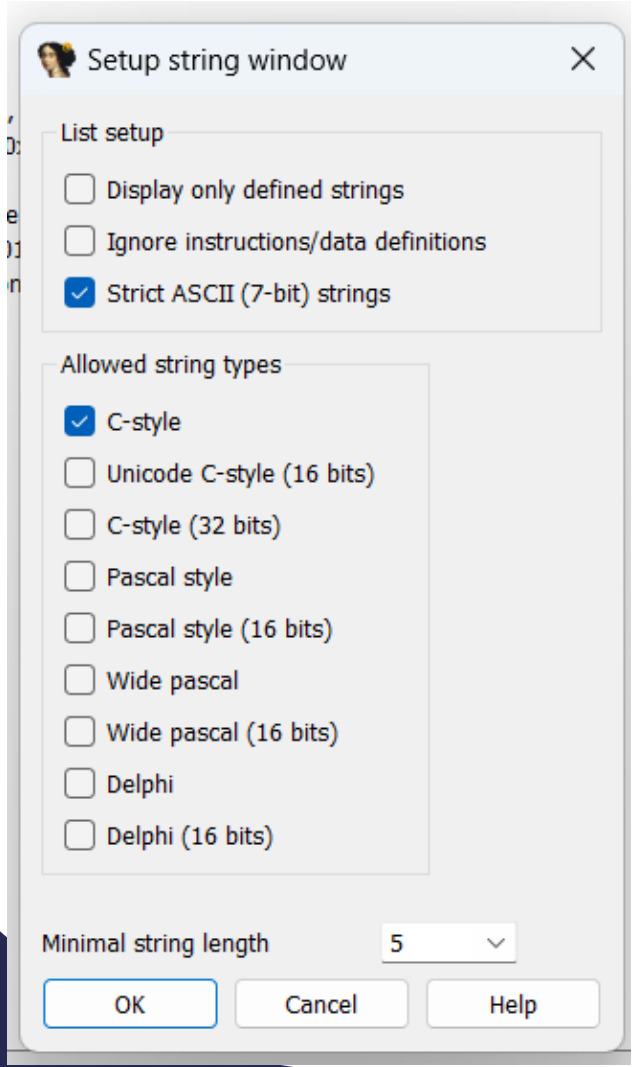
Address	Length	Type	String
.rdata:004040...	00000012	C	Lqydolg#Sdvvzrug$
.rdata:004040...	00000012	C	Sdvvzrug#RN$$$#=,
.rdata:004040...	00000019	C	IOLI Crackme Level 0x03\n
.rdata:004040...	0000000B	C	Password:
.rdata:004040...	0000002D	C	w32_sharedptr->size == sizeof(W32_EH_SHARED)
.rdata:004040...	0000003C	C	../../gcc-3.4.5-20060117-1/gcc/config/i386/w32-shared-ptr.c
.rdata:004041...	00000027	C	GetAtomNameA (atom, s, sizeof(s)) != 0

String setup window

Address	Length	Type	String
.rdata:004040...	00000012	C	Lqydolg#Sdvvzrug$
.rdata:004040...	00000012	C	Sdvvzrug#RN$$$#=,
.rdata:004040...	00000019	C	IOLI Crackme Level 0x03\n
.rdata:004040...	0000000B	C	Password:
.rdata:004040...	0000002D	C	w32_sharedptr->size == sizeof(W32_EH_SHARED)
.rdata:004040...	0000003C	C	../../gcc-3.4.5-20060117-1/gcc/config/i386/w32-shared-ptr.c
.rdata:004041...	00000027	C	GetAtomNameA (atom, s, sizeof(s)) != 0

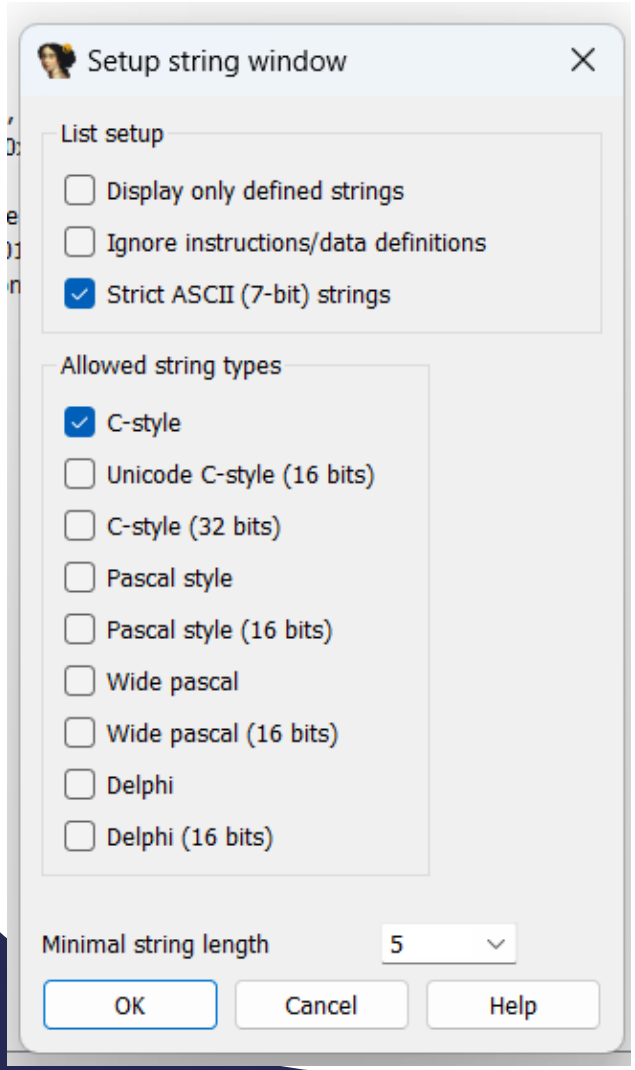



ddress
Length
Type
String
.rdata:004040... 00000012 C Lqydolg#Sdvvzrug$
.rdata:004040... 00000012 C Sdvvzrug#RN$$$#=,
.rdata:004040... 00000019 C IOLI Crackme Level 0
.rdata:004040... 0000000B C Password:
.rdata:004040... 0000002D C w32_sharedptr->size
.rdata:004040... 0000003C C ../../gcc-3.4.5-20060
.rdata:004041... 00000027 C GetAtomNameA (aton
Setup string window
List setup
Display only defined strings
Ignore instructions/data definitions
Strict ASCII (7-bit) strings
Allowed string types
C-style
Unicode C-style (16 bits)
C-style (32 bits)
Pascal style
Pascal style (16 bits)
Wide pascal
Wide pascal (16 bits)
Delphi
Delphi (16 bits)
Minimal string length
5
OK
Cancel
Help



Display only defined strings

- This option restricts the Strings window to displaying only named string data items that have been automatically created by IDA or manually created by the user.
- With this option selected, all other options are disabled, and IDA will not automatically scan for additional string content



Ignore instructions/data definitions

If checked, IDA will ignore instruction/data definitions and will try to treat them as strings.

Using this option allows IDA to

(1) see strings that may be embedded in the code portion of a binary and have been mistakenly converted into instructions or

(2) to see strings within data that may be formatted as something other than a string (such as an array of bytes or integers).

This option will also lead to the generation of many junk strings

The Names window provides a summary listing of all of the global names within a binary.

- A name is nothing more than a symbolic description given to a program virtual address.
- Names can be sorted alphabetically or in virtual address order (either ascending or descending).
- The Names window is useful for rapidly navigating to known locations within a program listing.

Name	Address	Public
_abort	**00402DF0**	**P**
__assert	00402E00	P
ExitProcess(x)	**00402E10**	**P**
SetUnhandledExceptionFilter(x)	**00402E20**	**P**
ReleaseSemaphore(x,x,x)	**00402E30**	**P**
InterlockedDecrement(x)	**00402E40**	**P**
WaitForSingleObject(x,x)	**00402E50**	**P**
CreateSemaphoreA(x,x,x,x)	**00402E60**	**P**
Sleep(x)	**00402E70**	**P**
InterlockedIncrement(x)	**00402E80**	**P**
GetAtomNameA(x,x,x)	**00402E90**	**P**
AddAtomA(x)	**00402EA0**	**P**
FindAtomA(x)	**00402EB0**	**P**
___reg_frame_ctor	00402EC0	
___dereg_frame_dtor	00402ED0	
___CTOR_LIST__	00402EE0	P
__DTOR_LIST__	00402EEC	P
__CRT_glob	00403000	P
__fmode	00403010	P
p.0	00403020	
dw2_once.1	00403030	
eh_globals_once.4	00403038	
__data_end__	00403040	P
Format	00404000	
aLqydolgSdvvzru	00404004	
aSdvvzrugRn	00404016	
aIoliCrackmeLev	00404028	
aPassword	00404041	

The coding scheme is summarized below:

F A regular function. These are functions that IDA does not recognize as library functions.

L A library function. IDA recognizes library functions through the use of signature-matching algorithms. If a signature does not exist for a given library function, the function will be labeled as a regular function instead.

I An imported name, most commonly a function name imported from a shared library.

The difference between this and a library function is that no code is present for an imported name, while the body of a library function will be present in the disassembly.

C Named code. These are named program instruction locations that IDA does not consider to be part of any function. This is possible when IDA finds a name in a program's symbol table but never sees a call to the corresponding program location.

D Data. Named data locations typically represent global variables.

A String data. This is a referenced data location containing a sequence of characters that conform to one of IDA's known string data types, such as a null-terminated ASCII C string

Symbols associate locations inside the file (e.g. addresses of functions or variables) with textual names (usually the names used in the original source code). The part of the file storing this association is commonly called *symbol table*. Symbols can be stored in the file itself or separately.

- If a location is named in the program’s symbol table, IDA adopts the name from the symbol table. If no symbol table entry is available for a given program location, IDA generates a default name for use in the disassembly.
- When IDA chooses to name a location, the virtual address of the location is combined with a prefix that indicates what type of location is being named.
- Incorporating the virtual address into a generated name ensures that all generated names will be unique, as no two locations can share the same virtual address.

Some of the more common prefixes used for autogenerated names include these:

- **sub_xxxxxx** A subroutine at address
- **xxxxxx loc_xxxxxx** An instruction location at address
- **xxxxxx byte_xxxxxx** 8-bit data at location
- **xxxxxx word_xxxxxx** 16-bit data at location
- **xxxxxx dword_xxxxxx** 32-bit data at location
- **xxxxxx unk_xxxxxx** Data of unknown size at location xxxxxx

The Segments window displays a summary listing of the segments present in the binary file.

- Note that what IDA terms segments are most often called sections when discussing the structure of binary files.
- Do not confuse the use of the term segments in this manner with the memory segments associated with CPUs that implement a segmented memory architecture.
- Information presented in the window includes the segment name, start and end addresses, and permission flags. The start and end addresses represent the virtual address range to which the program sections will be mapped at runtime.

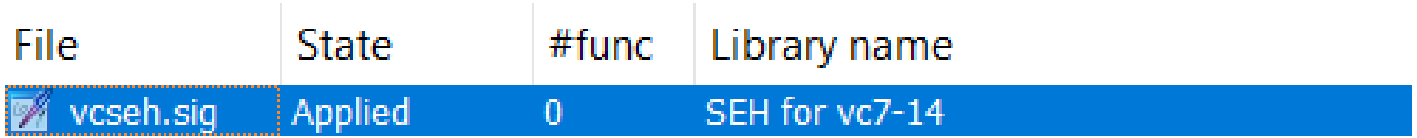
Name	Start	End	R	W	X	D	L	Align	Base	Type	Class	AD	es	ss	ds	fs	gs
.text	00401000	00403000	R	.	X	.	L	para	0001	public	CODE	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.data	00403000	00404000	R	W	.	.	L	para	0002	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.rdata	00404000	00405000	R	.	.	.	L	para	0003	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.bss	00405000	00406000	R	W	.	.	L	para	0004	public	BSS	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
.idata	00406000	00407000	R	W	.	.	L	para	0005	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF

Name	Start	End	R	W	X	D	L
.text	00401000	00403000	R	.	X	.	L
.data	00403000	00404000	R	W	.	.	L
.rdata	00404000	00405000	R	.	.	.	L
.bss	00405000	00406000	R	W	.	.	L
.idata	00406000	00407000	R	W	.	.	L

Align	Base	Type	Class	AD	es	ss	ds	fs	gs
para	0001	public	CODE	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0002	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0003	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0004	public	BSS	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF
para	0005	public	DATA	32	0000	0000	0002	FFFFF...	FFFFFFFFFFFFFFFF

The Signatures Window IDA makes use of an extensive library of signatures for identifying known blocks of code.

- Signatures are also used to categorize functions as known library functions inserted by a compiler or as functions added to the binary as a result of static linking.
- When IDA identifies library functions for you, you can focus more of your effort on the code that IDA did not recognize (which is probably far more interesting to you than reverse engineering the inner workings of printf).
- The Signatures window is used to list the signatures that IDA has already matched against the open binary file.

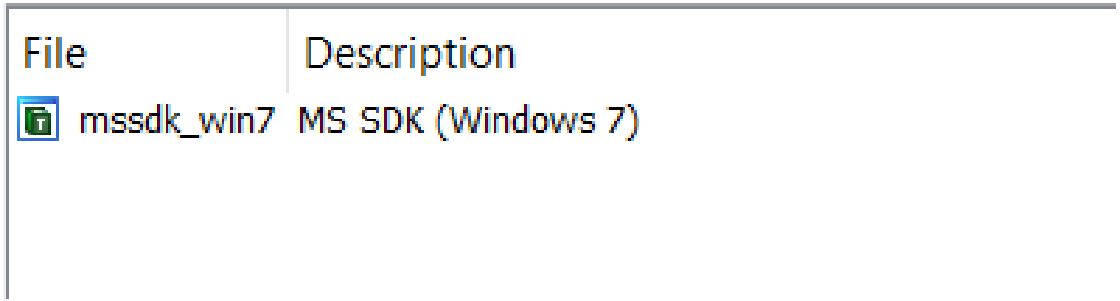


This example indicates that IDA has applied the vc32rtf signatures (from /sigs) against the binary and, in doing so, has been able to recognize 501 functions as library functions. That's 501 functions that you will not need to reverse engineer!

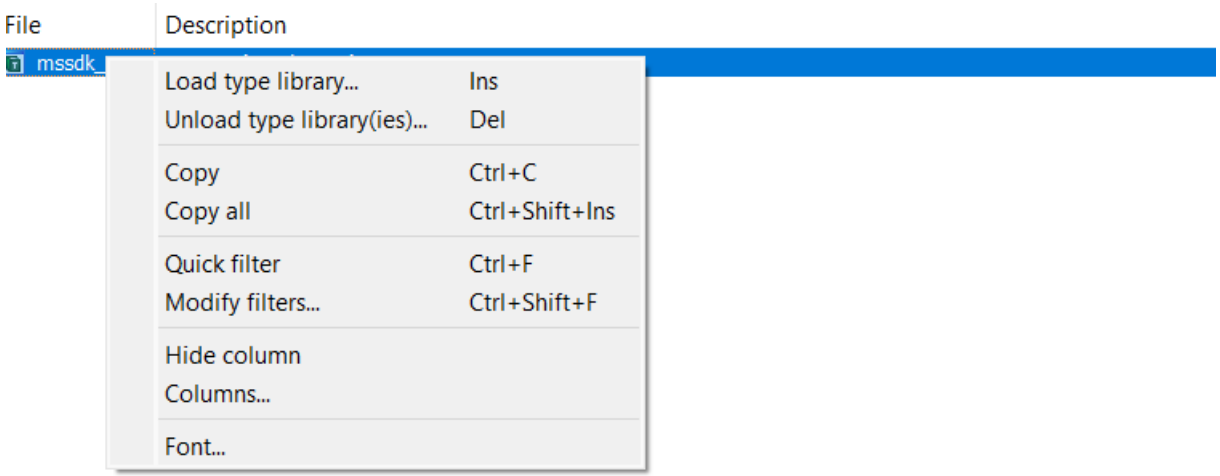
File	State	#func	Library name
vc32rtf	Applied	501	Microsoft VisualC 2-8/net runtime

The Type Libraries Window

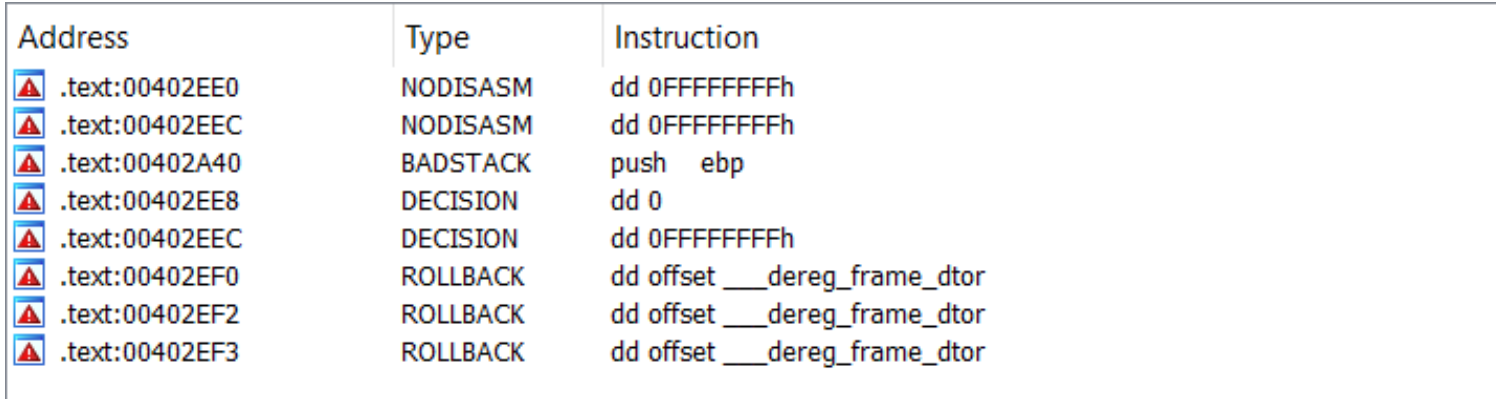
- Type libraries represent IDA's accumulated knowledge of predefined datatypes and function prototypes gleaned from header files included with most popular compilers.
- By processing header files, IDA understands the datatypes that are expected by common library functions and can annotate your disassemblies accordingly



- Here the user can load and unload standard type libraries.
- The standard type libraries contain type definitions from the standard C header supplied with compilers.



The Problems window is IDA's way of informing you of any difficulties that it has encountered in disassembling a binary and how it has chosen to deal with those difficulties.



Address	Type	Instruction
.text:00402EE0	NODISASM	dd 0FFFFFFFFh
.text:00402EEC	NODISASM	dd 0FFFFFFFFh
.text:00402A40	BADSTACK	push ebp
.text:00402EE8	DECISION	dd 0
.text:00402EEC	DECISION	dd 0FFFFFFFFh
.text:00402EF0	ROLLBACK	dd offset ___dereg_frame_dtor
.text:00402EF2	ROLLBACK	dd offset ___dereg_frame_dtor
.text:00402EF3	ROLLBACK	dd offset ___dereg_frame_dtor

Each problem is characterized by

(1) the address at which the problem occurs,

(2) the type of problem encountered, and

(3) the instruction present at the problem location.

In this example, we see a BOUNDS problem. A BOUNDS problem occurs when the destination of a call or jump either can't be determined (as in this example, since the value of eax is unknown to IDA) or appears to lie outside the range of virtual addresses in a program.

Address	Type	Instruction
.text:0040104C	BOUNDS	call eax
.text:004010B0	BOUNDS	call eax
.text:00401108	BOUNDS	call eax
.text:00401350	BOUNDS	call dword ptr [eax]
.text:004012A0	DECISION	push ebp
.text:004012D0	DECISION	push ebp
.text:00401560	DECISION	jmp ds:__set_app_type
.text:004015F8	DECISION	dd 0FFFFFFFFh
.text:004015FC	DECISION	dd 0

The following problems may occur:

- NOOFFSET Cannot find offset base
- NONAME Cannot find name
- NOFORCED Cannot find alternative string for an operand
- NOCMT Cannot find comment
- NOREF Cannot find references
- INDIRJMP Indirect execution flow
- NODISASM Cannot disassemble
- ALREADY Already data or code
- BOUNDS Execution flows beyond limits
- OVERFLOW Too many lines
- BADSTACK Failed to trace the value of the stack pointer
- LOOKHERE Attention! Probably erroneous situation
- DECISION Decision to convert to instruction/data is made by IDA
- ROLLBACK The decision made by IDA was wrong and rolled back
- COLISION FLAIR collision: the function with the given name already exists
- SIGFNREF FLAIR match indecision: reference to function expected